

Streszczenie

Niniejsze sprawozdanie opisuje projekt prototypu rozproszonego asystenta głosowego, który został zrealizowany w ramach przedmiotu *Systemy Mikroprocesorowe i Wbudowane*. Celem projektu było stworzenie funkcjonalnego rozwiązania typu Proof of Concept (PoC). System emuluje działanie komercyjnych głośników inteligentnych, takich jak Amazon Alexa czy Google Home, ze szczególnym uwzględnieniem integracji z infrastrukturą inteligentnego domu za pośrednictwem protokołu MQTT.

Architektura systemu obejmuje dwa kluczowe węzły:

- **Węzeł satelitarny (klient):** Działa na platformie Raspberry Pi Zero 2 W pod kontrolą systemu Raspberry Pi OS. Odpowiada za ciągłe nasłuchiwanie słowa budzącego (wake word), nagrywanie komendy użytkownika oraz odtwarzanie generowanych odpowiedzi.
- **Węzeł centralny (host):** Uruchomiony na maszynie stacjonarnej o większej mocy obliczeniowej. Zajmuje się zadaniami wymagającymi dużych zasobów: automatycznym rozpoznawaniem mowy (STT), przetwarzaniem języka naturalnego (NLP) za pomocą lokalnego dużego modelu językowego (LLM) oraz syntezą mowy (TTS).

Komunikacja między węzłami opiera się na protokole MQTT i jest zgodna ze specyfikacją standardu Hermes (wykorzystywanego m.in. w ekosystemie Rhasspy).

W fazie prototypu wykorzystano platformę Raspberry Pi Zero 2 W ze względu na jej uniwersalność i łatwy dostęp do bibliotek. Docelowo planowana jest jednak migracja modułu satelitarnego na mikrokontroler **Seeed XIAO ESP32-S3**. Układ ten charakteryzuje się znacznie mniejszymi wymiarami oraz sprzętową integracją interfejsów Wi-Fi i Bluetooth, co umożliwi pełną miniaturyzację urządzenia.

Opracowane rozwiązanie wykazuje wysoki potencjał wdrożeniowy w obszarze technologii asystencyjnych i opiekuńczych. Intuicyjny interfejs głosowy może stanowić kluczowe ułatwienie w codziennym funkcjonowaniu osób starszych lub o ograniczonej mobilności. W przyszłości planowane jest rozszerzenie funkcjonalności o integrację z narzędziem OpenClaw, co pozwoli asystentowi na autonomiczne przeszukiwanie zasobów sieciowych w czasie rzeczywistym oraz automatyzację zarządzania codziennymi zadaniami (np. organizacja kalendarza, generowanie i wysyłanie wiadomości e-mail).

1 Wstęp i cel projektu

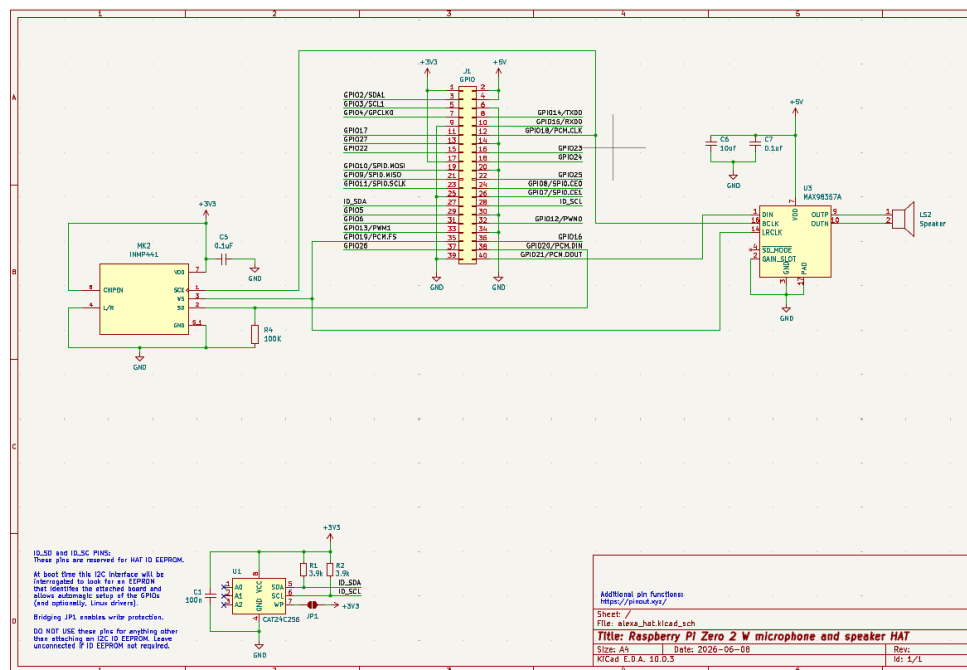
Dynamiczny rozwój koncepcji Internetu Rzeczy (IoT) oraz rosnąca wydajność i dostępność kompaktowych platform obliczeniowych umożliwiają implementację zaawansowanych systemów wbudowanych, realizujących funkcje zastrzeżone dotychczas dla zamkniętych rozwiązań komercyjnych. Głosowe interfejsy użytkownika (VUI) stanowią dziś jeden z fundamentalnych komponentów nowoczesnych ekosystemów Smart Home, oferując naturalną i bezdotykową kontrolę nad urządzeniami domowymi.

Głównym celem tego projektu było zaprojektowanie oraz implementacja autorskiego, lokalnego asystenta głosowego dedykowanego do obsługi **języka Polskiego**. Kluczowym założeniem projektowym było oparcie architektury wyłącznie na oprogramowaniu open-source oraz lokalnie uruchamianych modelach sztucznej inteligencji. Pozwoliło to na pełną niezależność od zewnętrznych dostawców usług chmurowych, gwarantując maksymalną prywatność i bezpieczeństwo przetwarzanych danych użytkownika.

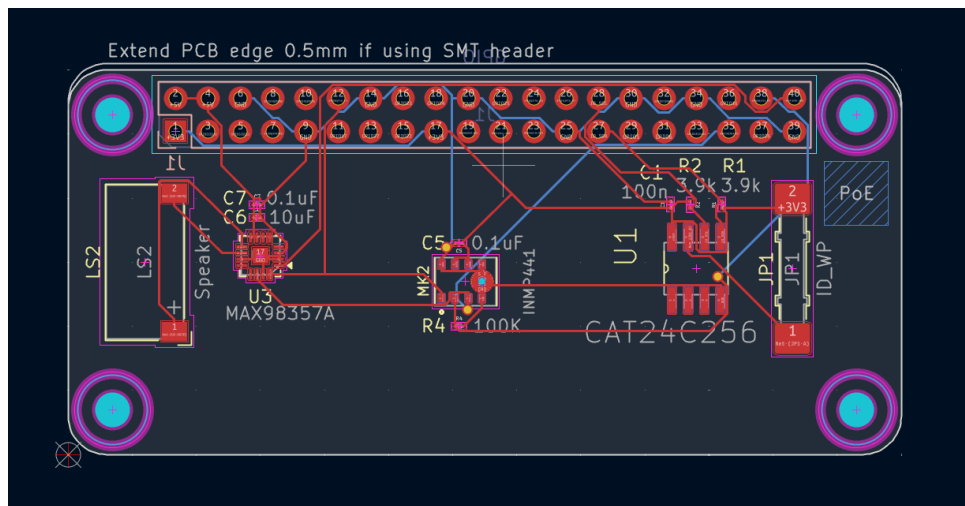
Projektowany system realizuje kompletny, sekwencyjny potok przetwarzania danych (pipeline), obejmujący następujące etapy:

1. Lokalna detekcja słowa budzącego (*Wake Word Detection*),
2. Rejestracja i buforowanie polecenia audio,
3. Zamiana mowy na tekst (*Speech-to-Text* – STT),
4. Analiza kontekstu i wygenerowanie odpowiedzi przez lokalny model LLM,
5. Zamiana tekstu na mowę (*Text-to-Speech* – TTS),
6. Transmisja zwrotna i odtworzenie syntezywanego komunikatu audio.

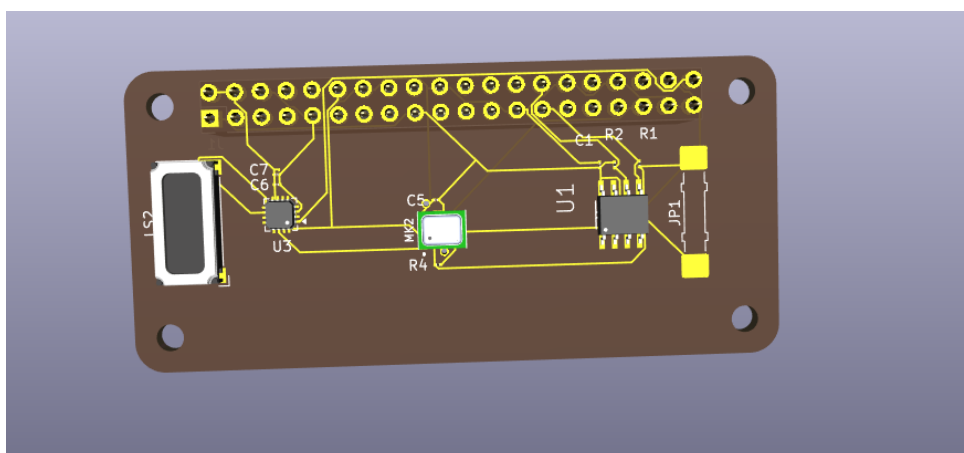
2 Schemat, PCB i wizualizacja 3D satelity



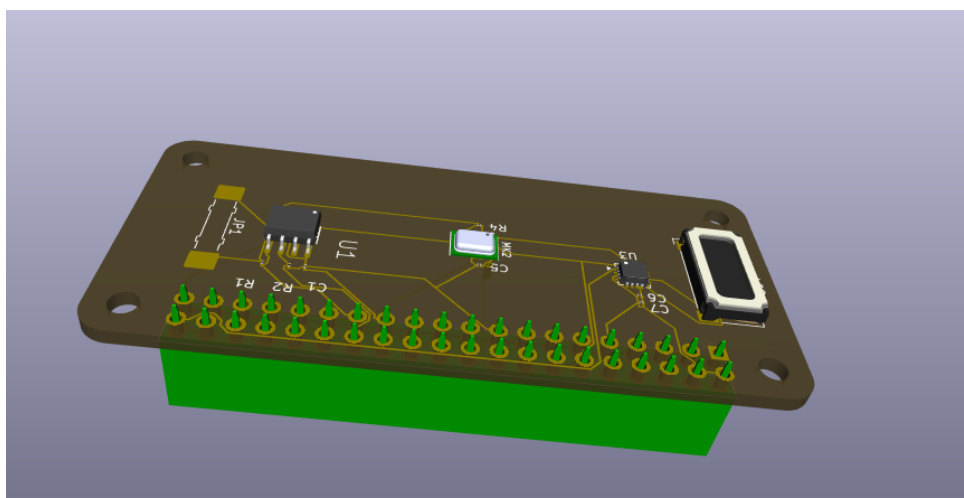
Rysunek 1: Schemat satelity



Rysunek 2: Projekt płytki PCB satelity

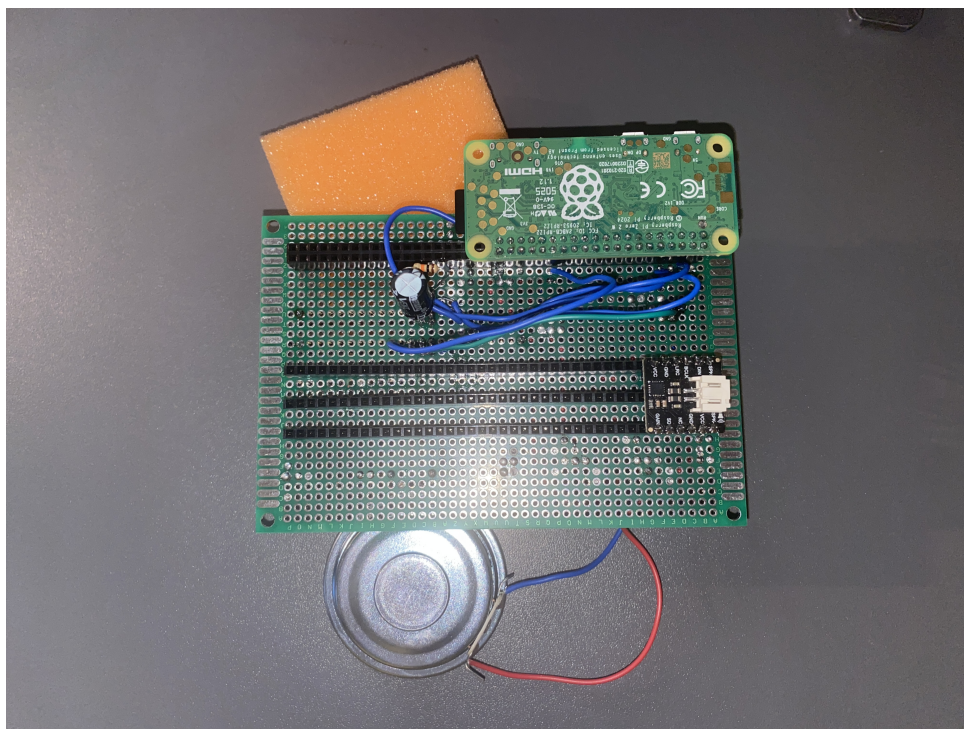


Rysunek 3: Wizualizacja 3D PCB satelity - rzut pierwszy

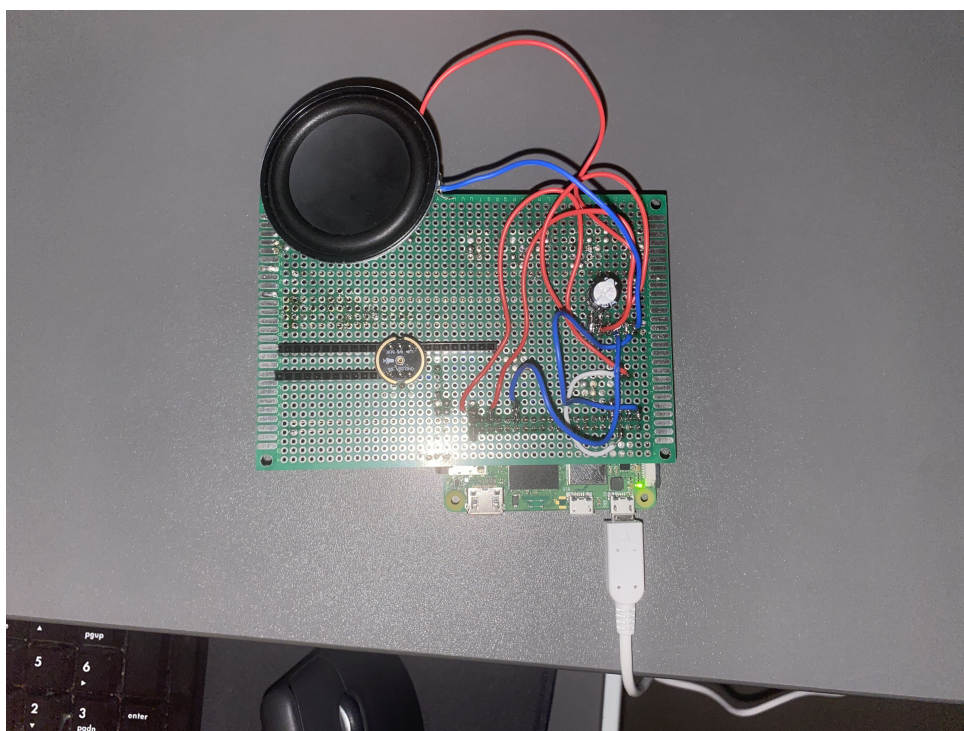


Rysunek 4: Wizualizacja 3D PCB satelity - rzut drugi

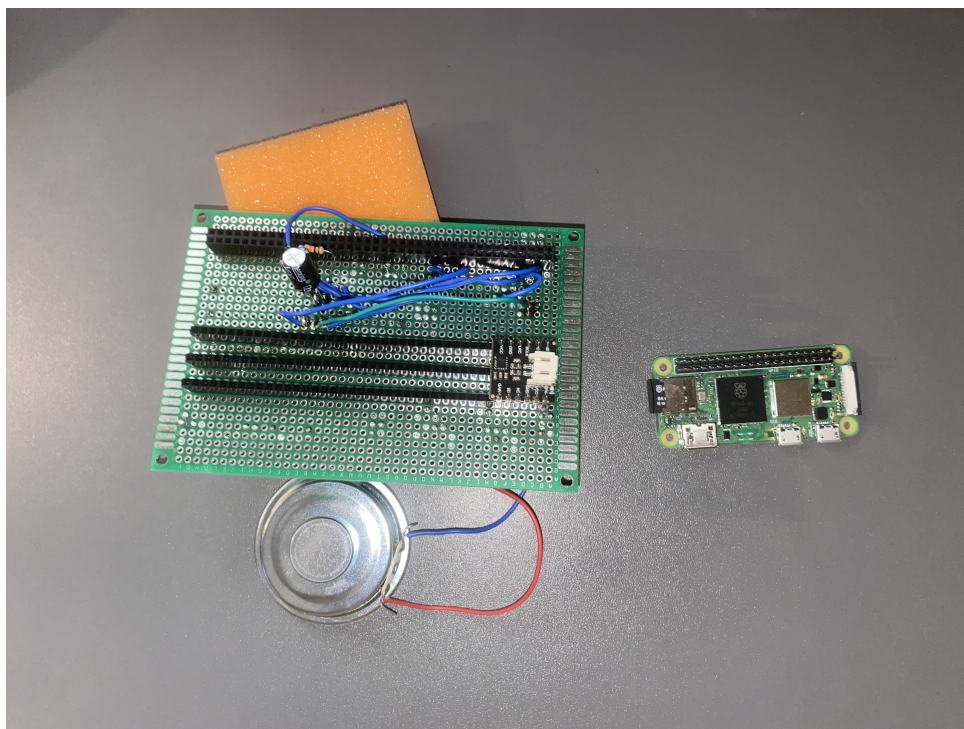
3 Fizyczny prototyp satelity



Rysunek 5: Satelita na płytce prototypowej - tył



Rysunek 6: Satelita na płytce prototypowej - front



Rysunek 7: Rozdzielony moduł satelity oraz RP Zero 2 W

4 Architektura systemu

4.1 Przegląd ogólny

Projektowany system charakteryzuje się architekturą rozproszoną, złożoną z dwóch logicznych węzłów komunikujących się za pośrednictwem centralnego brokera wiadomości. Asymetryczny podział zadań pozwala na optymalne zarządzanie zasobami sprzętowymi: energochłonne oraz wymagające dużej mocy obliczeniowej procesy (takie jak obsługa dużego modelu językowego LLM, systemu rozpoznawania mowy Whisper oraz syntezy TTS) są w pełni oddelegowane do wydajnego węzła centralnego (Hosta). Dzięki temu satelita realizuje wyłącznie lekkie zadania, związane z obsługą wejścia-wyjścia audio oraz detekcją frazy kluczowej.

4.2 Protokół komunikacji MQTT / Hermes

Wymiana danych pomiędzy poszczególnymi komponentami systemu opiera się na protokole MQTT (*Message Queuing Telemetry Transport*) – lekkim i efektywnym standardzie przesyłania wiadomości opartym na modelu subskrypcji (*publish-subscribe*), powszechnie stosowanym w ekosystemach Internetu Rzeczy (IoT).

W celu zapewnienia modułowości i standaryzacji danych, struktura tematów (topików) MQTT została oparta na specyfikacji **Hermes Protocol**, wykorzystywanej m.in. przez otwartoźródłową platformę asystentką Rhasspy. Implementacja tego standardu umożliwia łatwą migrację lub integrację poszczególnych modułów z innymi systemami zgodnymi z tą konwencją w przyszłości.

Do realizacji potoku przetwarzania danych w systemie wykorzystywane są następujące, kluczowe topiki MQTT:

- `hermes/hotword/default/detected`
Nadawca: Satelita | *Odbiorca:* Host
 Opublikowany automatycznie w momencie lokalnego wykrycia i zweryfikowania słowa budzącego przez moduł `openwakeword`.
- `hermes/asr/startListening` / `hermes/asr/stopListening`
Nadawca: Host/Satelita | *Odbiorca:* Moduł nagrywający
 Służą do precyzyjnego sterowania oknem czasowym rejestracji polecenia głosowego użytkownika.
- `hermes/asr/textCaptured`
Nadawca: Host (Whisper.cpp) | *Odbiorca:* Moduł zarządzający
 Zawiera zdekodowany tekst (wynik transkrypcji mowy na tekst), gotowy do przekazania do warstwy przetwarzania języka naturalnego.
- `hermes/nlu/query`
Nadawca: Moduł zarządzający | *Odbiorca:* Host (Ollama)
 Przekazuje przetworzone zapytanie tekstowe bezpośrednio do lokalnego modelu LLM w celu wygenerowania odpowiedzi.
- `hermes/nlu/intentNotRecognized` lub `hermes/tts/say`
Nadawca: Host (Ollama) | *Odbiorca:* Host (Piper TTS)
 Przenosi finalną odpowiedź tekstową wygenerowaną przez model językowy, stanowiąc sygnał wejściowy dla syntezy mowy.
- `hermes/audioServer/satellite/playBytes`
Nadawca: Host | *Odbiorca:* Satelita
 Służy do binarnego przesyłania surowego strumienia audio (w formacie WAV), generowanego przez syntezy TTS, bezpośrednio do podsystemu nagłośnienia satelity w celu jego odtworzenia.

5 Opis modułów urządzenia satelitarnego

5.1 Główna pętla zdarzeń – `satellite_main.py`

Plik `satellite_main.py` pełni funkcję centralnego modułu zarządzającego (orkiestratora) oprogramowania satelity. W celu zapewnienia optymalnej responsywności systemu, zaimplementowano w nim architekturę wielowątkową. Synchronizacja trzech niezależnych, równoległe uruchomionych procesów realizowana jest za pomocą obiektów `threading.Event`, działających jako binarne flagi stanu:

- `wake_word_loop` – wątek odpowiedzialny za ciągłe monitorowanie sygnału z mikrofonu w celu wykrycia frazy aktywacyjnej. Jest on tymczasowo blokowany na czas rejestracji polecenia użytkownika.
- `command_loop` – wątek aktywowany asynchronicznie po detekcji słowa budzącego, odpowiedzialny za rejestrację właściwego polecenia głosowego i jego transmisję do modułu transkrypcji STT.
- `send_intent_loop` – wątek odpowiedzialny za publikację zdekodowanego tekstu w postaci ustrukturyzowanego komunikatu MQTT do warstwy NLU/LLM na hoście.

Mechanizm bezpiecznej i bezkonfliktowej synchronizacji wątków ilustruje poniższy fragment kodu:

```
1 import threading
2
3 # Inicjalizacja flag synchronicznych
4 wake_word_active = threading.Event()
5 record_command_active = threading.Event()
6
7 # Stan początkowy: aktywna wyłącznie detekcja frazy kluczowej
8 wake_word_active.set()
9 record_command_active.clear()
10
11 # Procedura obsługi zdarzenia po wykryciu słowa budzącego
12 wake_word_active.clear() # Wstrzymanie detekcji Wake Word
13 record_command_active.set() # Rejestracja polecenia
```

Wywołanie metody `.set()` odblokowuje wątki oczekujące na dane zdarzenie (metoda `.wait()`), natomiast `.clear()` przywraca stan blokady. Taka implementacja skutecznie eliminuje hazardy systemowe, uniemożliwiając jednoczesny zapis do strumienia audio przez moduł detekcji słowa budzącego i moduł rejestracji poleceń.

5.2 Detekcja słowa budzącego – `wake_word.py`

Zadaniem tego modułu jest ciągła analiza strumienia wejściowego audio z mikrofonu w poszukiwaniu frazy aktywacyjnej 'Alexa'. Implementacja bazuje na bibliotece `openwakeword`, wykorzystującej zoptymalizowane modele głębokie w formacie ONNX.

W celu zapewnienia natywnej kompatybilności z modelem, strumień audio skonfigurowano według następujących parametrów:

- Częstotliwość próbkowania (RATE): 16000 Hz
- Rozmiar ramki (CHUNK): 1280 próbek (Interwał czasowy ~ 80 ms)
- Format danych (FORMAT): `paInt16` (16-bitowy strumień PCM)
- Liczba kanałów (CHANNELS): 1 (mono)

Kluczowy fragment pętli detekcyjnej przedstawia się następująco:

```
1 import numpy as np
2
3 # Odczyt surowych danych z bufora karty dźwiękowej
4 audio_data = stream.read(CHUNK, exception_on_overflow=False)
5 audio_array = np.frombuffer(audio_data, dtype=np.int16)
6
7 # Inferencja modelu openwakeword
8 prediction = model.predict(audio_array)
9 score = prediction['alexa_v0.1']
10
11 if score > 0.9:
12     return True
```

Zastosowanie rygorystycznego progu klasyfikacji (`score > 0.9`) pozwoliło zminimalizować współczynnik fałszywych aktywacji (*False Positives*) w środowisku o podwyższonym poziomie szumu tła. Przed przekazaniem danych do klasyfikatora, bufor audio konwertowany jest do zoptymalizowanej tablicy NumPy o typie danych `np.int16`.

5.3 Nagrywanie polecenia z detekcją ciszy – `record_command.py`

Po odebraniu sygnału wyzwolenia z modułu `wake_word.py`, system przechodzi w stan rejestracji komendy użytkownika. Moduł `record_command.py` realizuje algorytm dynamicznego wykrywania końca wypowiedzi na podstawie analizy aktywności głosowej za pomocą silnika VAD (*Voice Activity Detection*).

Do filtracji sygnału wykorzystano bibliotekę `webrtcvad`:

```
1 import webrtcvad
2
3 vad = webrtcvad.Vad()
4 vad.set_mode(3) # Tryb 3: najbardziej agresywna filtracja
                  szumów i ciszy
```

Wybór najwyższego stopnia agresywności (tryb 3) sprawia, że algorytm traktuje nawet krótkie, bezgłośnie przerwy w artykulacji jako ciszę. Zdefiniowanie parametru granicznego `MAX_SILENCE_FRAMES = 50` oznacza, że nagrywanie zostaje automatycznie przerwane po upływie 1,5 sekundy nieprzerwanej ciszy ($50 \times 30 \text{ ms} = 1500 \text{ ms}$).

Algorytm przetwarzania ramki audio przebiega w następujących krokach:

1. Cykliczny odczyt pakietów audio o rozmiarze `CHUNK = 480` próbek (ramka 30 ms przy próbkowaniu 16 kHz).
2. Ewaluacja obecności sygnału mowy w ramce za pomocą metody `vad.is_speech()`.
3. Jeśli zostanie wykryta aktywność głosowa, a stan nagrywania był nieaktywny – następuje inicjalizacja bufora wyjściowego.
4. W przypadku braku detekcji mowy, inkrementowany jest licznik ramek ciszy.
5. Po przekroczeniu wartości `MAX_SILENCE_FRAMES` proces rejestracji kończy się.
6. Skumulowane w pamięci RAM ramki audio są serializowane i zapisywane na dysku jako plik `command.wav`.

5.4 Transmisja danych audio do modułu STT – `send_command.py`

Wygenerowany plik `command.wav` przesyłany jest asynchronicznie za pomocą protokołu HTTP (metoda POST) do instancji serwera `Whisper.cpp` uruchomionego na maszynie centralnej. `Whisper.cpp` stanowi wysoce zoptymalizowaną (napisaną w C/C++) implementację modelu automatycznego rozpoznawania mowy (OpenAI Whisper), udostępniającą lokalny interfejs REST API.

Konstrukcja żądania i parametry inferencji zdefiniowane zostały w następujący sposób:


```

1 import requests
2
3 payload = {
4     'temperature': '0.0', # deterministyczne greedy search
5     'language': 'pl',      # Jawne wskazanie języka polskiego
6     'response_format': 'json'
7 }
8
9 files = {
10     'file': ('command.wav', open('command.wav', 'rb'), 'audio/wav')
11 }
12
13 response = requests.post(
14     f'http://{host_ip}:8080/inference',
15     files=files,
16     data=payload
17 )

```

Wymuszenie parametru `temperature = 0.0` eliminuje losowość (wariancję) podczas próbkowania tokenów, co gwarantuje powtarzalność transkrypcji dla tych samych próbek dźwiękowych.

6 Opis modułów jednostki centralnej (Hosta)

6.1 Główna pętla hosta – `host_run.py`

Skrypt `host_run.py` stanowi główny punkt wejścia oprogramowania uruchomionego na hoście. Odpowiada za subskrypcję i obsługę topiku `hermes/nlu/intentNotRecognized`. Wybór tego tematu wynika z założeń projektowych: zapytania, które nie pasują do żadnych sztywno zdefiniowanych reguł intencji, są traktowane jako zapytania ogólne i przekazywane do obsługi przez lokalny duży model językowy.

Architektura potoku przetwarzania zdarzeń została zrealizowana w sposób asynchroniczny z wykorzystaniem biblioteki `asyncio`. Sekwencja obsługi przychodzącego komunikatu MQTT przebiega według następującego schematu:

1. Dekodowanie ładunku (payload) JSON i wyodrębnienie pola `input`, zawierającego tekstową postać polecenia użytkownika.
2. Asynchroniczne przekazanie zapytania do lokalnego modelu LLM za pomocą metody `llmrequest.chat()`.
3. Przekierowanie wygenerowanej odpowiedzi tekstowej do modułu syntezy mowy `tts.synthesize()`.
4. Publikacja wynikowego pliku binarnego WAV na odpowiednim topiku komunikacyjnym satelity poprzez procedurę `respond.send()`.

Zastosowanie programowania asynchronicznego (`asyncio`) pozwala na równoległe wykonywanie nieblokujących operacji, takich jak wyświetlanie animowanego wskaźnika stanu oczekiwania (`display_waiting`), przy jednoczesnym zachowaniu pełnej reaktywności pętli zdarzeń MQTT. Kluczowe wywołania bibliotek zewnętrznych (interfejsy serwera Olama oraz Piper TTS) zostały spakowane w natywne koutyny (`async def`).

6.2 Komunikacja z modelem LLM – llmrequest.py

Moduł `llmrequest.py` implementuje warstwę komunikacyjną z lokalnym API serwera Ollama. Narzędzie to umożliwia w pełni suwerenną i odizolowaną od sieci zewnętrznej inferencję wielkich modeli językowych.

W projekcie wykorzystano model **Bielik-11B-v2.3-Instruct** (stworzony przez polski zespół SpeakLeash) w wersji skwantowanej do formatu **Q4_K_M**. Wybór tego modelu podyktowany był jego wysoką optymalizacją pod kątem gramatyki i semantyki języka polskiego. Zastosowanie kwantyzacji 4-bitowej pozwoliło na redukcję zapotrzebowania na pamięć VRAM/RAM z początkowych ~ 22 GB do około 7 GB przy marginalnym spadku precyzji odpowiedzi, co umożliwiło stabilne uruchomienie systemu na maszynie wyposażonej w 16 GB pamięci operacyjnej.

Za wysłanie zapytania odpowiada poniższy fragment kodu:

```
1 from ollama import AsyncClient
2
3 messages = [
4     {'role': 'system', 'content': sysmsg},
5     {'role': 'user', 'content': usrmsg}
6 ]
7
8 response = await AsyncClient().chat(
9     model=model,
10    messages=messages,
11    options={'num_predict': 128} # Ograniczenie dlugosci
12                                odpowiedzi
13 )
14 return response['message']['content']
```

W konfiguracji pominięto zaawansowane tryby wieloetapowego rozumowania (*chain-of-thought*), ponieważ w systemach asystenckich priorytetem jest minimalizacja czasu reakcji (opóźnienia *Time-to-First-Token*), a nie rozbudowane wnioskowanie logiczne. Odpowiednio skonstruowany komunikat systemowy (`sysmsg`) wymusza na modelu generowanie odpowiedzi zwięzłych, sformułowanych w pierwszej osobie liczby pojedynczej, oraz całkowite pomijanie znaków interpunkcyjnych czy formatowania Markdown, co stanowi warunek konieczny dla poprawnego działania syntezy mowy.

6.3 Synteza mowy – tts.py

Konwersja tekstu wyjściowego na sygnał mowy realizowana jest lokalnie przez moduł Piper TTS – szybki i wydajny syntezytor oparty na architekturze neuronowej VITS. W projekcie wykorzystano oficjalny model głosowy `pl_PL-gosia-medium`. Charakteryzuje się on optymalnym kompromisem pomiędzy naturalnością brzmienia syntetyzowanej mowy (prozodia) a niskim kosztem obliczeniowym inferencji.

Konfiguracja procesu syntezy opiera się na następujących parametrach struktury `SynthesisConfig`:

- Głośność (`VOLUME = 1.0`): Pełna skala sygnału, brak programowej normalizacji.
- Tempo mowy (`LENGTH_SCALE = 1.0`): Standardowa prędkość artykulacji.
- Wariancja szumu (`NOISE_SCALE = 0.667`): Domyślna wartość określająca zmienność fonetyczną i naturalność brzmienia.

- Wariancja długości fonemów (NOISE_W_SCALE = 0.8): Parametr kontrolujący rytm oraz intonację wypowiedzi.

Generowanie pliku audio realizowane jest za pomocą następującej sekwencji:

```

1 import wave
2 from piper import PiperVoice, SynthesisConfig
3
4 # Załadowanie modelu z akceleracją sprzętową CUDA
5 voice = PiperVoice.load(voice_path, use_cuda=True)
6 syn_config = SynthesisConfig(volume=VOLUME, length_scale=
    LENGTH_SCALE)
7
8 with wave.open(filename, 'wb') as wav_file:
9     voice.synthesize_wav(text, wav_file, syn_config)

```

Ustawienie flagi `use_cuda=True` przenosi obliczenia tensorowe bezpośrednio na procesor graficzny (GPU) firmy NVIDIA. Pozwala to na osiągnięcie współczynnika RTF (*Real-Time Factor*) znacznie poniżej wartości 1.0, co oznacza, że synteza dźwięku trwa wielokrotnie krócej niż czas jego późniejszego odtwarzania.

6.4 Transmisja odpowiedzi do satelity – `respond.py`

Po zakończeniu generowania pliku audio, moduł `respond.py` odpowiada za dostarczenie danych binarnych z powrotem do urządzenia satelitarne. Transmisja realizowana jest asynchronicznie poprzez odczyt surowych bajtów pliku WAV i ich bezpośrednią publikację na dedykowanym topiku `hermes/audioServer/satellite/playBytes`.

```

1 with open(filename, 'rb') as f:
2     audio_bytes = f.read()
3
4 # Publikacja strumienia binarnego z flagą oczekiwania
5 msg_info = client.publish(topic, payload=audio_bytes, qos=0)
6 msg_info.wait_for_publish()

```

Wywołanie metody `wait_for_publish()` blokuje wykonanie wątku do momentu, aż broker MQTT potwierdzi poprawne przyjęcie pakietu danych. W projekcie zdecydowano się na poziom jakości usług QoS 0 (*at most once*), co jest standardem przy przesyłaniu strumieni medialnych w sieciach lokalnych, gdzie priorytetem jest minimalizacja narzutu protokołu nadmiarowego. Po odebraniu komunikatu przez klienta na Raspberry Pi, surowe bajty przekazywane są bezpośrednio do podsystemu ALSA za pomocą systemowego narzędzia `aplay`, co skutkuje natychmiastową emisją dźwięku przez głośnik satelity.

7 Konfiguracja systemu

7.1 Konfiguracja hosta – config.ini

Host wykorzystuje standardowy plik INI parsowany przez moduł `configparser`. Plik rozdzielony jest na sekcje odpowiadające poszczególnym komponentom systemu:

- [LLM.REQUEST] – model, komunikat systemowy dla LLM
- [TTS] – ścieżka do modelu głosu, parametry syntezy, nazwa pliku wyjściowego
- [RESPOND] – adres IP brokera MQTT, port, topik dla audio
- [HOSTRUN] – adres IP i port brokera MQTT dla odebrania poleceń

7.2 Konfiguracja satelity – config.txt / read_config.py

Satelita używa własnego, uproszczonego mechanizmu konfiguracji opartego na pliku tekstowym w formacie KLUCZ=WARTOSC. Moduł `read_config.py` parsuje ten plik linia po linii, wyszukując zadany aparat i zwracając wartość po znaku równości.

```
1 def get_from_config(parameter=''):
2     with open('config.txt', 'r') as config_file:
3         for line in config_file:
4             if parameter in line:
5                 parts = line.split('=')
6                 return parts[1].strip(' \r\n\"')
```

Metoda `strip()` usuwa białe znaki, znaki nowej linii oraz cudzysłowy z wartości, co zapewnia poprawne działanie niezależnie od formatu zapisu w pliku konfiguracyjnym.

8 Platforma docelowa – Seeed XIAO ESP32-S3

Obecny prototyp został zrealizowany na Raspberry Pi Zero 2 W z systemem Raspberry Pi OS, co pozwoliło na szybkie prototypowanie dzięki pełnej dostępności Pythona i bibliotek Linux. Platforma ta jest jednak nadmiarowa dla roli satelity, która wymaga jedynie: nagrywania audio, detekcji słowa budzącego, komunikacji Wi-Fi/MQTT i odtwarzania audio.

Docelowa platforma produkcyjna to moduł Seeed XIAO ESP32-S3, który oferuje istotne przewagi nad Raspberry Pi Zero dla tego zastosowania:

Tabela 1: Porównanie platform sprzętowych dla węzła satelitarnego

Cecha	RPi Zero 2 W (Prototyp)	XIAO ESP32-S3 (Cel)
Wymiary PCB	65 x 30 mm	21 x 17.5 mm
Pobór mocy (idle)	~120 mA @ 5V	~20 mA @ 3.3V
Język programowania	Python	C++ (Arduino/IDF)
Koszt	~70 PLN	~35 PLN

Migracja na ESP32-S3 będzie wymagała przepisania modułów satelity z Pythona na C++ z użyciem frameworku ESP-IDF lub Arduino. Główne wyzwania techniczne to: implementacja klienta MQTT (biblioteka `PubSubClient` lub `esp-mqtt`), obsługa mikrofonu przez interfejs I2S, uruchomienie lekkiego modelu detekcji słowa budzącego (np. *microWakeWord*) oraz streaming audio PCM po Wi-Fi do hosta.

9 Wyniki i obserwacje

9.1 Zmierzone czasy odpowiedzi (sieć lokalna)

Pomiary wykonane dla typowego polecenia (ok. 5-10 słów):

- **Detekcja słowa budzącego:** < 200 ms (`openwakeword` na RPi Zero)
- **Nagrywanie polecenia (VAD):** 2-4 sekundy (zależne od długości wypowiedzi)
- **Transkrypcja `Whisper.cpp`:** 1-3 sekundy (model `medium.pl` na GPU hosta)
- **Odpowiedź LLM (Bielik 11B Q4):** 3-8 sekund (zależne od długości odpowiedzi)
- **Synteza Piper TTS:** < 2 sekundy (z akceleracją CUDA)
- **Przesył audio MQTT:** < 200 ms
- **Łączny czas odpowiedzi:** 7-16 sekund

9.2 Zidentyfikowane ograniczenia PoC

- Brak obsługi historii konwersacji – każde zapytanie jest niezależne (*stateless*).
- Model Bielik wykazuje tendencję do generowania interpunkcji mimo instrukcji w `sysmsg`.
- Detekcja VAD bywa zbyt czuła na hałas otoczenia (tryb agresywności 3 może przedwcześnie kończyć nagrywanie przy dłuższych pauzach).
- Brak mechanizmu timeoutu w przypadku braku odpowiedzi hosta.

10 Wnioski

Projekt stanowi pomyślnie zrealizowany dowód koncepcji (PoC) lokalnego asystenta głosowego działającego w pełni offline, bez przysyłania danych do chmury. Architektura oparta na MQTT i protokole Hermes okazała się elastyczna i łatwa w rozbudowie – dodanie nowych komponentów sprowadza się do subskrypcji odpowiednich topików.

Kluczowym wyzwaniem okazało się uzyskanie akceptowalnego czasu odpowiedzi przy użyciu lokalnego modelu LLM klasy 11B parametrów. Wyniki wskazują, że dla zastosowań produkcyjnych warto rozważyć mniejsze, wyspecjalizowane modele (np. 3B-7B) lub zastosowanie dedykowanego sprzętu wspierającego inferencję (NPU, mocniejsze GPU).

Przejsie na platformę Seeed XIAO ESP32-S3 w wersji finalnej pozwoli na znaczną redukcję kosztów, poboru mocy i rozmiarów urządzenia satelitarnego, przy zachowaniu pełnej funkcjonalności systemu. Architektura hosta pozostaje niezmienna – modyfikacje dotyczą wyłącznie warstwy sprzętowej i programowej satelity.

11 Literatura i źródła

1. Rhasspy Voice Assistant - Hermes Protocol: <https://rhasspy.readthedocs.io/>
2. OpenWakeWord - open-source wake word detection: <https://github.com/dscripka/openWakeWord>
3. Whisper.cpp - C++ port of OpenAI Whisper: <https://github.com/ggerganov/whisper.cpp>
4. Ollama - local LLM runner: <https://ollama.com>
5. SpeakLeash Bielik - polski model LLM: <https://huggingface.co/speakleash>
6. Piper TTS - neural text-to-speech: <https://github.com/rhasspy/piper>
7. Eclipse Mosquitto MQTT Broker: <https://mosquitto.org>
8. Seeed XIAO ESP32-S3 datasheet: https://wiki.seeedstudio.com/xiao_esp32s3_getting_started/
9. WebRTC VAD: <https://github.com/wiseman/py-webrtcvad>
10. Eclipse Paho MQTT Python: <https://github.com/eclipse/paho.mqtt.python>